

MÓDULO IV — DESARROLLO FULLSTACK

# SESIÓN 3

## Pruebas Unitarias, de Componentes y de API

*Informe de ejecución de los tres laboratorios*

|              |                                              |
|--------------|----------------------------------------------|
| Estudiante   | Albert Omar Araya Cory                       |
| Repositorio  | github.com/albrtaraya/fullstack-lab-1-1ci-cd |
| Rama         | feature/pruebas-vitest                       |
| Pull Request | #5 — Abierto, pipeline en verde              |
| Suite        | 19 pruebas en 3 archivos                     |
| Modalidad    | Individual                                   |
| Fecha        | 09 de septiembre de 2026                     |

### Insignias obtenidas

 Cazador de Bugs (10 pts) ·  Triple Verde (15 pts)

## Resumen de ejecución

La Sesión 3 construye la pirámide de pruebas sobre el proyecto: pruebas unitarias de funciones puras, pruebas de componente que simulan interacción real del usuario, y pruebas de API. El tercer laboratorio cierra el ciclo con una cacería de bugs guiada por pruebas.

| Laboratorio | Nombre                                           | Tiempo | Estado   |
|-------------|--------------------------------------------------|--------|----------|
| 1           | Configurar Vitest y pruebas unitarias            | 40 min | Cumplido |
| 2           | Prueba de componente con React Testing Library   | 40 min | Cumplido |
| 3           | Cacería de Bugs: prueba primero, corrige después | 25 min | Cumplido |

## Punto de partida: qué había y qué faltaba

El prerequisite de la sesión (pipeline de CI de la Sesión 2 en verde) estaba cumplido. Sin embargo, al revisar el proyecto apareció una condición que condiciona todo el trabajo de esta sesión y conviene declarar por delante:

| Lo que pedía la guía                        | Lo que había en el proyecto                                 |
|---------------------------------------------|-------------------------------------------------------------|
| Una función pura que probar                 | Ninguna: el sitio no tenía lógica propia                    |
| Un componente de React con un formulario    | Ninguno: el proyecto es Astro estático, sin React instalado |
| Un endpoint de API que probar con Supertest | Ninguno: el proyecto no tiene backend                       |

**Decisión tomada:** en lugar de crear archivos que existan solo para que las pruebas tengan algo que mirar, se construyó una funcionalidad pequeña pero real — una lista de tareas — que atraviesa las tres capas y queda efectivamente montada en la página del sitio. Las pruebas verifican así código que el proyecto usa, no andamiaje desechable.

## Arquitectura resultante

|                                    |                                    |                          |
|------------------------------------|------------------------------------|--------------------------|
| src/utils/validaciones.js          | Funciones puras                    | -> pruebas unitarias     |
| src/components/FormularioTarea.jsx | Componente React                   | -> pruebas de componente |
| src/components/ListaTareas.jsx     | Componente React (usa el anterior) |                          |
| src/server/app.js                  | API Express                        | -> pruebas de API        |
| src/pages/index.astro              | Monta <ListaTareas client:load />  |                          |

Las tres capas comparten las mismas validaciones: la regla «un título no puede ser solo espacios en blanco» vive en un único lugar y se comporta igual en el formulario y en la API.

## Laboratorio 1 — Configurar Vitest y pruebas unitarias

*Objetivo: instalar Vitest y escribir las primeras pruebas unitarias sobre una función pura, siguiendo el patrón Arrange-Act-Assert.*

### Instalación y configuración

```
$ npm install -D vitest

# vitest.config.js
import { defineConfig } from 'vitest/config';

export default defineConfig({
  test: {
    globals: true,
    environment: 'node',
  },
});
```

Se agregó el script "test": "vitest run" al package.json.

### Funciones puras creadas

El proyecto no tenía ninguna función pura, así que se creó el módulo src/utils/validaciones.js. Son puras porque su resultado depende únicamente de sus parámetros: no consultan base de datos, no modifican variables externas ni hacen peticiones de red.

| Función                        | Qué hace                                                   |
|--------------------------------|------------------------------------------------------------|
| esCorreoValido(correo)         | Indica si la cadena tiene formato de correo válido         |
| contarTareasPendientes(tareas) | Cuenta las tareas cuyo campo completada es falso           |
| normalizarTitulo(titulo)       | Recorta espacios de los extremos y colapsa los internos    |
| esTituloValido(titulo)         | Indica si el título tiene contenido real, no solo espacios |

### Pruebas escritas: caso esperado y caso límite

Cada función se probó con al menos un caso esperado (datos normales) y un caso límite (lista vacía, cadena mal formada, valor que no es texto). Los bugs reales casi siempre viven en los casos límite.

```
// src/utils/validaciones.test.js

describe('esCorreoValido', () => {
  it('acepta un correo con formato válido', () => {
    // Arrange
    const correo = 'ana@ejemplo.com';
    // Act
    const resultado = esCorreoValido(correo);
    // Assert
    expect(resultado).toBe(true);
  });

  it('rechaza un correo sin arroba', () => { ... });
  it('rechaza un correo sin dominio', () => { ... });
  it('no se rompe cuando el valor no es una cadena', () => { ... });
});
```

```
});  
  
describe('contarTareasPendientes', () => {  
  it('cuenta solo las tareas no completadas', () => {  
    const tareas = [  
      { completada: true },  
      { completada: false },  
      { completada: false },  
    ];  
    expect(contarTareasPendientes(tareas)).toBe(2);  
  });  
  
  it('devuelve 0 cuando la lista está vacía', () => {  
    expect(contarTareasPendientes([])).toBe(0);  
  });  
});
```

## Resultado

```
$ npx vitest run
```

```
RUN   v5.0.0
```

```
Test Files  1 passed (1)  
Tests       10 passed (10)  
Duration    501ms
```

## Checkpoint del Laboratorio 1

- ✓ Vitest está instalado y configurado en vitest.config.js.
- ✓ El script npm run test ejecuta las pruebas correctamente.
- ✓ Hay pruebas unitarias en verde: caso esperado y caso límite para cada función.

## Laboratorio 2 — Prueba de componente con React Testing Library

*Objetivo: escribir una prueba que simule una interacción real de usuario sobre un componente y verifique el resultado visible, no la implementación interna.*

### Instalación

```
$ npm install -D @testing-library/react @testing-library/jest-dom \
  @testing-library/user-event jsdom

# El proyecto no tenía React, así que también fue necesario:
$ npm install react react-dom @astrojs/react
$ npm install -D @vitejs/plugin-react
```

Se habilitó la integración de React en `astro.config.mjs` para que el componente pueda montarse en el sitio, no solo en las pruebas.

### Entorno jsdom

```
// vitest.config.js
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: ['./src/test/setup.js'],
  },
});

// src/test/setup.js
import '@testing-library/jest-dom';
```

### El componente probado

`FormularioTarea.jsx` reutiliza las validaciones puras del Laboratorio 1, de modo que la regla de negocio no se duplica:

```
export default function FormularioTarea({ onAgregar }) {
  const [titulo, setTitulo] = useState('');

  function manejarEnvio(e) {
    e.preventDefault();
    if (esTituloValido(titulo)) {
      onAgregar(normalizarTitulo(titulo));
      setTitulo('');
    }
  }

  return (
    <form onSubmit={manejarEnvio}>
      <label htmlFor="titulo-tarea">Nueva tarea</label>
      <input id="titulo-tarea" value={titulo}
        onChange={(e) => setTitulo(e.target.value)} />
      <button type="submit">Agregar</button>
    </form>
  );
}
```

```
);  
}
```

## La prueba simula al usuario, no inspecciona el estado

```
it('llama a onAgregar con el texto escrito por el usuario', async () => {  
  // Arrange  
  const onAgregar = vi.fn();  
  render(<FormularioTarea onAgregar={onAgregar} />);  
  const usuario = userEvent.setup();  
  
  // Act  
  const input = screen.getByLabelText('Nueva tarea');  
  await usuario.type(input, 'Comprar pan');  
  await usuario.click(screen.getByText('Agregar'));  
  
  // Assert  
  expect(onAgregar).toHaveBeenCalled('Comprar pan');  
});
```

Los elementos se buscan por su etiqueta y su texto visibles (`getByLabelText`, `getByText`), igual que los encontraría una persona usando un lector de pantalla. Nunca por clase CSS ni por el nombre de una variable interna: así la prueba sigue siendo válida aunque cambie el diseño del componente.

### Se escribieron cuatro pruebas de componente:

| Prueba                                              | Qué verifica                               |
|-----------------------------------------------------|--------------------------------------------|
| llama a onAgregar con el texto escrito              | El camino feliz de la interacción          |
| no llama a onAgregar si el campo está vacío         | Caso límite: campo sin contenido           |
| no llama a onAgregar si el título son solo espacios | Caso límite: contenido aparente pero vacío |
| limpia el campo después de agregar                  | El efecto visible tras el envío            |

## El componente quedó montado en el sitio real

`ListaTareas.jsx` envuelve al formulario, mantiene el estado y usa `contarTareasPendientes` para mostrar el contador. Se monta en `src/pages/index.astro` con `client:load`, de modo que las pruebas cubren código que el sitio realmente ejecuta.

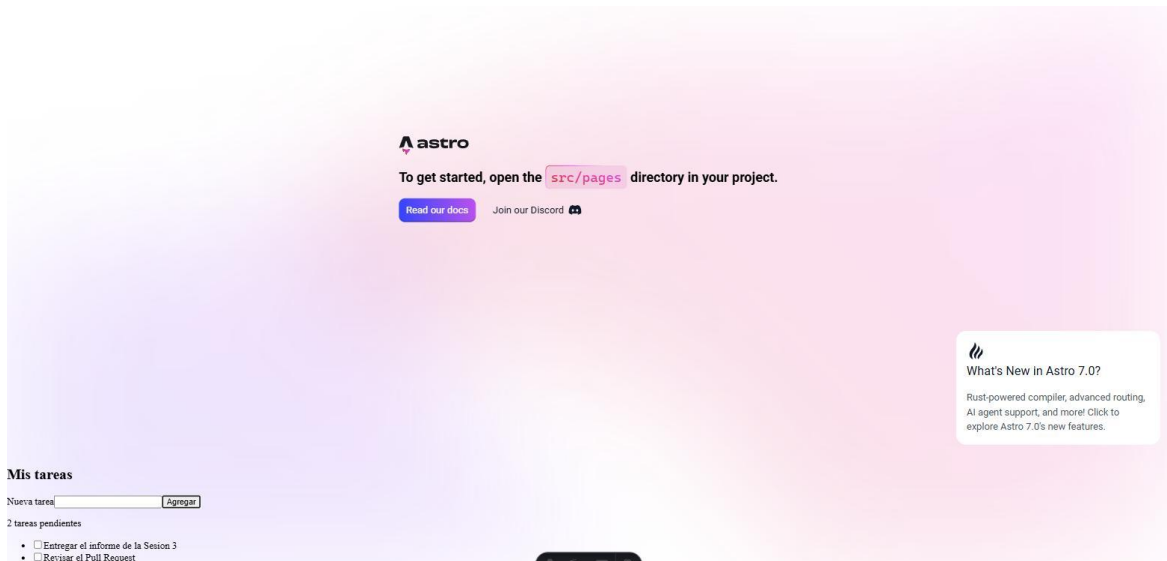


Figura 1. La lista de tareas funcionando en el sitio: dos tareas agregadas desde el formulario y el contador mostrando «2 tareas pendientes».

## Checkpoint del Laboratorio 2

- ✓ React Testing Library instalada y entorno jsdom configurado.
- ✓ Las pruebas simulan interacción real (escribir y hacer clic), no revisan estado interno.
- ✓ Las pruebas pasan en verde y no dependen del diseño visual del componente.

## Laboratorio 3 — Cacería de Bugs

*Objetivo: vivir el ciclo prueba-primero, corrige-después. Demostrar que el bug existe con una prueba que falla, y solo entonces corregirlo.*

**Ticket atendido: BUG 3 (API) — el endpoint POST /tareas acepta y guarda una tarea con título vacío o solo espacios.**

### Preparación: Supertest y la API

```
$ npm install -D supertest
$ npm install express
```

La instancia de Express se exporta **sin** llamar a `app.listen()`, para que Supertest la levante en un puerto efímero durante las pruebas y no haya un ECONNREFUSED contra un puerto real.

La API se creó en su estado defectuoso, tal como llegaría en un proyecto real: el endpoint guardaba cualquier título sin validarlo.

```
// src/server/app.js -- ESTADO INICIAL, CON EL BUG
app.post('/tareas', (req, res) => {
  const { titulo } = req.body ?? {};

  const nuevaTarea = { id: siguienteId++, titulo, completada: false };
  tareas.push(nuevaTarea);
  res.status(201).json(nuevaTarea);
});
```

### Ronda 1 — Reproducir el bug

Se escribieron tres pruebas nuevas que describen el comportamiento CORRECTO esperado, no el que el código tenía ese momento:

```
it('rechaza crear una tarea con titulo vacio', async () => {
  const res = await request(app).post('/tareas').send({ titulo: '' });

  expect(res.status).toBe(400);
});

it('rechaza crear una tarea cuyo titulo son solo espacios', async () => {
  const res = await request(app).post('/tareas').send({ titulo: ' ' });

  expect(res.status).toBe(400);
});

it('no guarda las tareas rechazadas en la lista', async () => {
  await request(app).post('/tareas').send({ titulo: ' ' });

  const res = await request(app).get('/tareas');
  const titulosVacios = res.body.filter((t) => !t.titulo || !t.titulo.trim());

  expect(titulosVacios).toHaveLength(0);
});
```

**Resultado de la ejecución: el pipeline de pruebas en ROJO.**

```
$ npx vitest run
```



```

> tests/tareas.test.js (5 tests | 3 failed) 85ms
  > API de tareas (5)
    x rechaza crear una tarea con titulo vacio 15ms
    x rechaza crear una tarea cuyo titulo son solo espacios 8ms
    x no guarda las tareas rechazadas en la lista 15ms

----- Failed Tests 3 -----

FAIL tests/tareas.test.js > API de tareas > rechaza crear una tarea con titulo vacio
AssertionError: expected 201 to be 400 // Object.is equality

- Expected
+ Received

- 400
+ 201

> tests/tareas.test.js:27:22
  27 |     expect(res.status).toBe(400);
    |                        ^

Test Files  1 failed | 2 passed (3)
Tests      3 failed | 16 passed (19)

```

El bug queda demostrado con evidencia objetiva: la API responde 201 Created donde debería responder 400 Bad Request, y las tareas vacías sí se estaban guardando (3 encontradas en la lista).

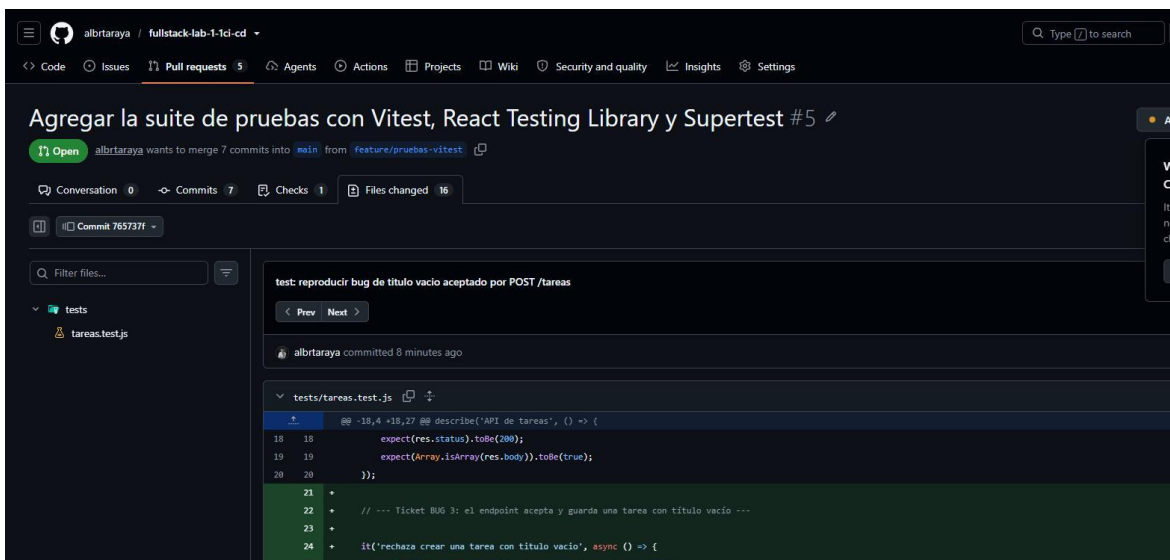


Figura 2. Commit 765737f: las tres pruebas que reproducen el bug, agregadas antes de tocar el código fuente.

## Ronda 2 — Corregir el código

La corrección se hizo en el archivo fuente, nunca debilitando la aserción de la prueba:

```

// src/server/app.js -- CORREGIDO
import { esTituloValido, normalizarTitulo } from '../utils/validaciones.js';

app.post('/tareas', (req, res) => {
  const { titulo } = req.body ?? {};

  if (!esTituloValido(titulo)) {
    return res.status(400).json({ error: 'El titulo es obligatorio' });
  }
});

```

```

}

const nuevaTarea = {
  id: siguienteId++,
  titulo: normalizarTitulo(titulo),
  completada: false,
};
tareas.push(nuevaTarea);
res.status(201).json(nuevaTarea);
});

```

La validación reutiliza la misma función pura que usa el formulario del Laboratorio 2, de modo que frontend y backend no pueden discrepar sobre qué título es aceptable.

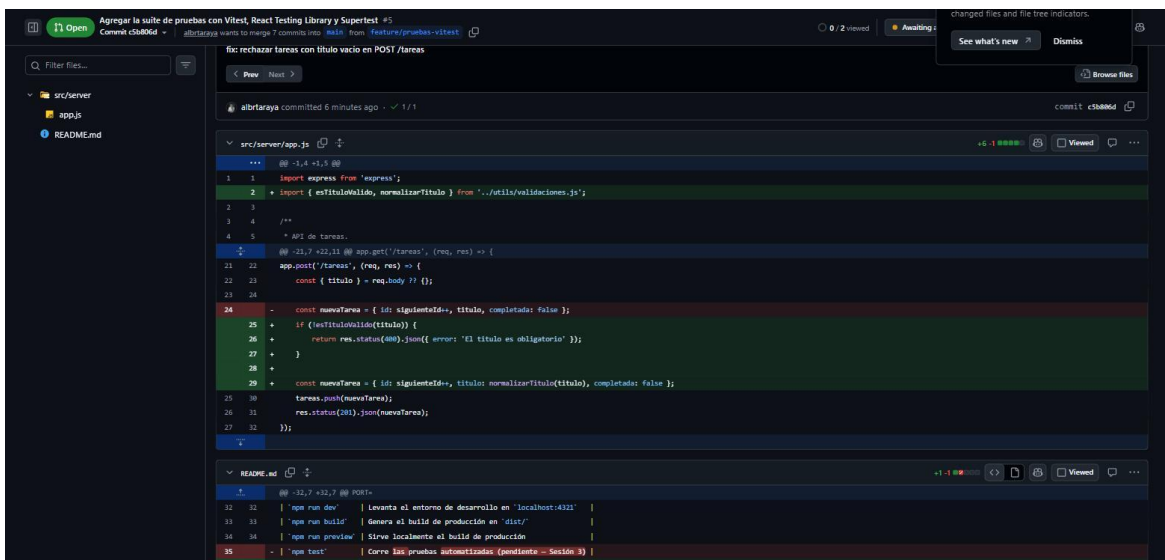


Figura 3. Commit c5b806d: la corrección en src/server/app.js. El archivo de prueba no se tocó.

## Ronda 3 — Verificación final: Triple Verde

Suite completa en una sola ejecución, con las tres capas de la pirámide:

```
$ npx vitest run --reporter=verbose
```

```

✓ tests/tareas.test.js > API de tareas > crea una tarea nueva 375ms
✓ tests/tareas.test.js > API de tareas > lista las tareas creadas 5ms
✓ tests/tareas.test.js > API de tareas > rechaza crear una tarea con titulo vacío 4ms
✓ tests/tareas.test.js > API de tareas > rechaza ... solo espacios 4ms
✓ tests/tareas.test.js > API de tareas > no guarda las tareas rechazadas 10ms
✓ src/utils/validaciones.test.js > esCorreoValido > acepta un correo válido 2ms
✓ src/utils/validaciones.test.js > esCorreoValido > rechaza sin arroba 0ms
✓ src/utils/validaciones.test.js > esCorreoValido > rechaza sin dominio 0ms
✓ src/utils/validaciones.test.js > esCorreoValido > no se rompe si no es cadena 0ms
✓ src/utils/validaciones.test.js > contarTareasPendientes > cuenta no completadas 0ms
✓ src/utils/validaciones.test.js > contarTareasPendientes > 0 con lista vacía 0ms
✓ src/utils/validaciones.test.js > normalizarTitulo > recorta los extremos 0ms
✓ src/utils/validaciones.test.js > normalizarTitulo > colapsa espacios internos 0ms
✓ src/utils/validaciones.test.js > esTituloValido > acepta contenido real 1ms
✓ src/utils/validaciones.test.js > esTituloValido > rechaza solo espacios 0ms
✓ src/components/FormularioTarea.test.jsx > llama a onAgregar con el texto 422ms
✓ src/components/FormularioTarea.test.jsx > no llama si el campo está vacío 44ms

```

```
✓ src/components/FormularioTarea.test.jsx > no llama si son solo espacios 122ms
✓ src/components/FormularioTarea.test.jsx > limpia el campo después 354ms

Test Files  3 passed (3)
Tests      19 passed (19)
```

| Capa de la pirámide | Archivo                                 | Pruebas | Estado |
|---------------------|-----------------------------------------|---------|--------|
| Unitaria            | src/utls/validaciones.test.js           | 10      | Verde  |
| Componente          | src/components/FormularioTarea.test.jsx | 4       | Verde  |
| API                 | tests/tareas.test.js                    | 5       | Verde  |
| Total               | 3 archivos                              | 19      | Verde  |

## El ciclo queda registrado en el historial

Los commits no se reescribieron: el orden demuestra que la prueba se escribió antes que la corrección.

| Commit  | Mensaje                                                        | Rol en el ciclo                |
|---------|----------------------------------------------------------------|--------------------------------|
| e103223 | feat: agregar API de tareas con pruebas de integracion         | API creada con el bug presente |
| 765737f | test: reproducir bug de titulo vacio aceptado por POST /tareas | FALLA                          |
| c5b806d | fix: rechazar tareas con titulo vacio en POST /tareas          | PASA                           |

## Checkpoint del Laboratorio 3

- ✓ Se escribió una prueba que reproduce el bug y se confirmó que falla.
- ✓ Se corrigió el código fuente, no la prueba.
- ✓ La suite completa corre en verde: unitarias, componente y API a la vez.

Insignias obtenidas: 🐛 Cazador de Bugs (10 pts) · 🟢 Triple Verde (15 pts)

## Hallazgo adicional: ESLint no revisaba los archivos .jsx

Al ejecutar el lint sobre el código React recién escrito, el pipeline devolvía verde sin reportar nada. La revisión en detalle mostró la causa:

```
$ npx eslint src/components/FormularioTarea.jsx
0:0 warning File ignored because no matching configuration was supplied
```

La configuración de ESLint heredada de la Sesión 2 no declaraba ninguna entrada para los archivos .jsx, de modo que ESLint los saltaba por completo. El step de lint del pipeline pasaba en verde sin haber leído una sola línea de los componentes: una falsa sensación de seguridad sobre todo el código nuevo.

Comprobación: se introdujo a propósito una variable sin usar en un componente y el lint seguía pasando. Tras agregar la entrada de configuración con `parserOptions.ecmaFeatures.jsx`, el mismo error sí se detecta:

```
$ npx eslint src/components/ListaTareas.jsx

53:7 error 'noUsada' is assigned a value but never used no-unused-vars

✖ 1 problem (1 error, 0 warnings)
```

También se declararon las globales de Vitest (`describe`, `it`, `expect`, `vi`) para los archivos de prueba, que de otro modo habrían disparado errores de `no-undef`.

## Estado del pipeline y del Pull Request

Todo el trabajo se integró en la rama `feature/pruebas-vitest` y se propuso mediante el Pull Request #5, siguiendo el flujo establecido en la Sesión 1: ningún cambio entra a `main` sin revisión.

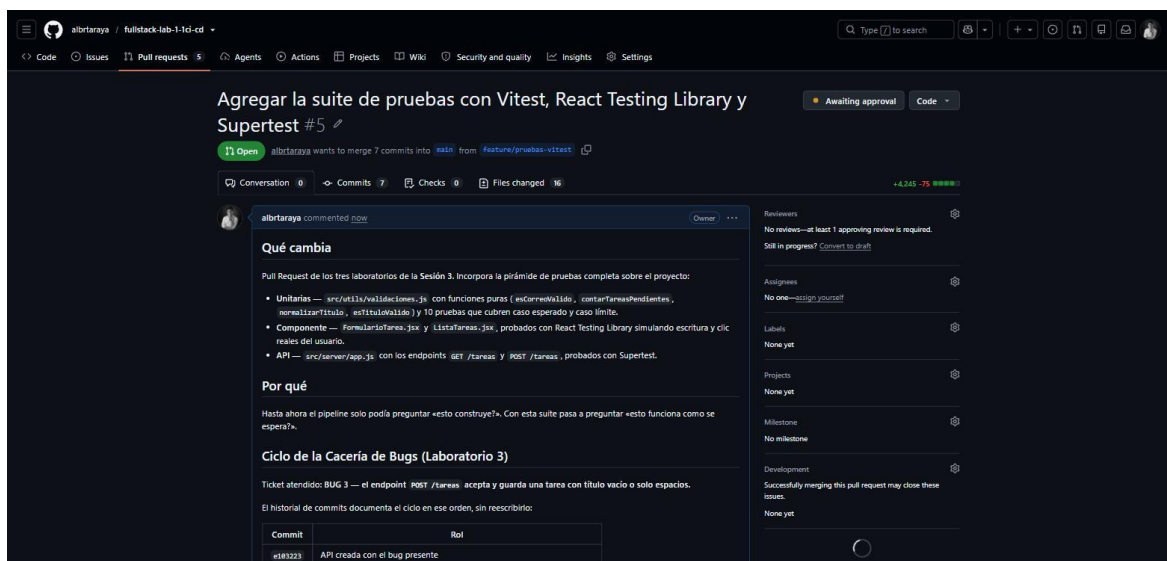


Figura 4. Pull Request #5 abierto, con la descripción documentando el ciclo de la cacería de bugs.

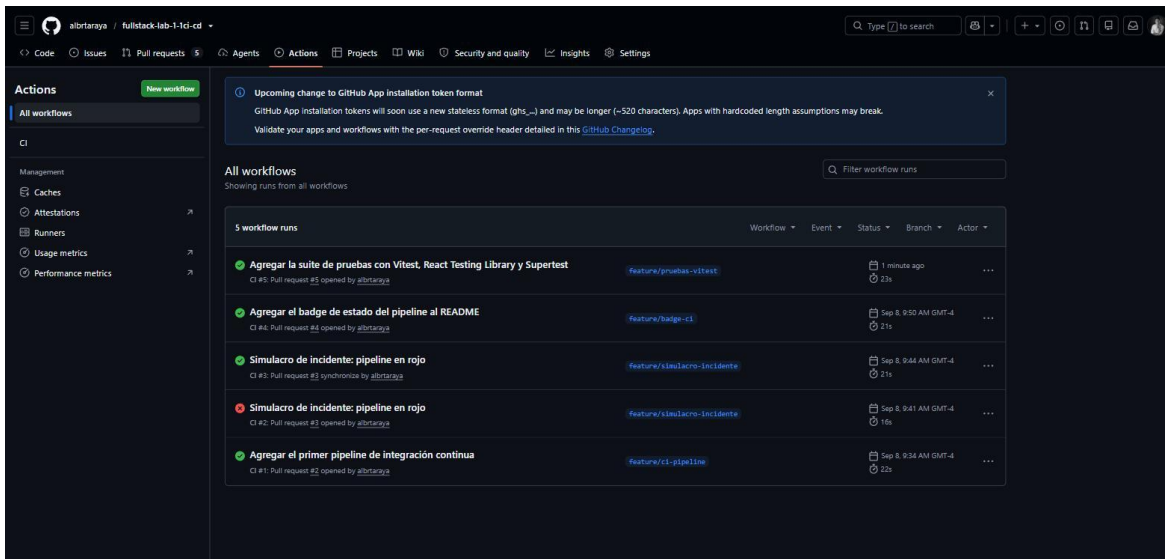


Figura 5. Historial de ejecuciones del pipeline. El run CI #5 corresponde a este Pull Request.

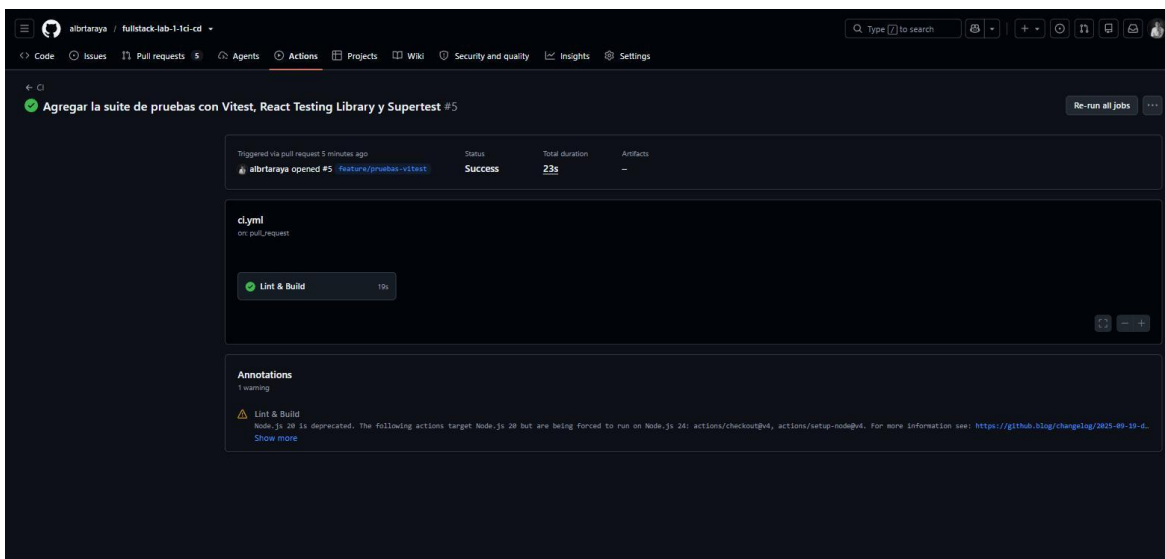


Figura 6. Run CI #5: Status «Success» en 23 segundos, con el job Lint & Build en verde sobre todo el código nuevo.

**Nota sobre el alcance del pipeline:** el workflow ejecuta `npm ci`, `npm run lint` y `npm run build`. Todavía no ejecuta `npm test`: conectar la suite de pruebas al pipeline para que un Pull Request quede bloqueado automáticamente si algo falla es precisamente el contenido de la Sesión 4, así que se deja para ahí en lugar de adelantarlos.

## Entregable final de la Sesión 3

| Requisito                                                                                        | Estado   |
|--------------------------------------------------------------------------------------------------|----------|
| Vitest configurado, con el script npm run test funcionando                                       | Cumplido |
| Al menos dos pruebas unitarias en verde sobre una función pura (caso esperado + caso límite)     | Cumplido |
| Al menos una prueba de componente con React Testing Library en verde, simulando interacción real | Cumplido |
| Al menos una prueba de API con Supertest en verde                                                | Cumplido |
| Un ciclo completo documentado de prueba en rojo → código corregido → prueba en verde             | Cumplido |
| La suite completa corre en verde con un solo comando (npx vitest run)                            | Cumplido |

## Sobre la revisión de los Pull Requests

Los cinco Pull Requests del repositorio están abiertos y con el pipeline en verde, pero siguen sin revisor asignado: el repositorio no tiene colaboradores y GitHub no permite que el autor se apruebe a sí mismo. Para cerrarlo hace falta invitar al compañero revisor o al docente desde Settings → Collaborators, y asignarlo en el panel «Reviewers» de cada Pull Request.

## Siguiente estación

Con la suite en verde, la Sesión 4 la extiende: pruebas end-to-end con Playwright, medición de cuánto código cubren las pruebas, y la conexión de todo esto al pipeline para que un Pull Request quede bloqueado automáticamente si algo falla.

## Anexo A — Índice de evidencias

| Archivo                          | Contenido                                         |
|----------------------------------|---------------------------------------------------|
| 01-pr-formulario.jpg             | Formulario del Pull Request #5 antes de crearlo   |
| 02-pr-abierto.jpg                | Pull Request #5 abierto con su descripción        |
| 03-commit-prueba-en-rojo.jpg     | Commit 765737f: las pruebas que reproducen el bug |
| 04-commit-correccion-a-verde.jpg | Commit c5b806d: la corrección del código fuente   |
| 05-actions-historial.jpg         | Historial de ejecuciones del pipeline             |
| 06-run-ci-success.jpg            | Run CI #5 con Status Success                      |
| 07-componente-funcionando.jpg    | La lista de tareas funcionando en el sitio        |

## Anexo B — Dependencias incorporadas

| Paquete                          | Para qué                                                     |
|----------------------------------|--------------------------------------------------------------|
| vitest                           | Ejecutor de pruebas                                          |
| @testing-library/react           | Renderizar y consultar componentes como lo haría un usuario  |
| @testing-library/jest-dom        | Matchers adicionales (toBeInTheDocument, toHaveValue)        |
| @testing-library/user-event      | Simular escritura y clics reales                             |
| jsdom                            | DOM simulado para las pruebas de componente                  |
| @vitejs/plugin-react             | Transformar JSX dentro de Vitest                             |
| supertest                        | Peticiones HTTP contra la app de Express sin abrir un puerto |
| react, react-dom, @astrojs/react | Soporte de React en el proyecto Astro                        |
| express                          | Servidor de la API de tareas                                 |

## Anexo C — Archivos creados y modificados

### CREADOS

|                                         |                         |
|-----------------------------------------|-------------------------|
| vitest.config.js                        | Configuración de Vitest |
| src/test/setup.js                       | Matchers de jest-dom    |
| src/utils/validaciones.js               | Funciones puras         |
| src/utils/validaciones.test.js          | 10 pruebas unitarias    |
| src/components/FormularioTarea.jsx      | Componente React        |
| src/components/FormularioTarea.test.jsx | 4 pruebas de componente |
| src/components/ListaTareas.jsx          | Componente contenedor   |
| src/server/app.js                       | API de tareas (Express) |
| tests/tareas.test.js                    | 5 pruebas de API        |

### MODIFICADOS

|                       |                               |
|-----------------------|-------------------------------|
| package.json          | Script test + dependencias    |
| astro.config.mjs      | Integración de React          |
| eslint.config.js      | Cobertura de archivos .jsx    |
| src/pages/index.astro | Monta <ListaTareas />         |
| README.md             | npm test ya no está pendiente |